

Übungs-Zusammenfassung Cybersicherheit

Prof. Dr.-Ing. Lucas Vincenzo Davi
Juniorprofessur für Informatik
und
M.Sc. Christian Niesler
wissenschaftlicher Mitarbeiter
Universität Duisburg-Essen

18. Juli 2025
Sommersemester 2024

Vorname: _____

Nachname: _____

Hinweis

Bei dieser Übungszusammenfassung handelt es sich um ein paar umstrukturierte Aufgaben aus den bisherigen Übungen. Ziel dieser kleinen Übungszusammenfassung ist es in der letzten Übung dieses Semester, Übungsinhalte zu wiederholen und evtl. Fragen zu Übungsaufgaben zu klären. Sie erhalten hiermit die Gelegenheit sich erneut mit den Übungsinhalten durch ähnlich aufgebaute Übungsaufgaben auseinander zu setzen.

1. Betriebsmodi einer Blockchiffre

Eine einfache Blockchiffre $d(\cdot)$ mit **6-Bit Blockgröße** [Block ist $(b_1, b_2, b_3, b_4, b_5, b_6)$] entschlüsselt, indem die Chiffre eine schlüsselabhängige Bit-Permutation durchführt. Für *einen* bestimmten Schlüssel, der für diese Aufgabe angewendet wird, führt die Blockchiffre folgende Entschlüsselung aus:

$$d(b_1, b_2, b_3, b_4, b_5, b_6) = (b_6, b_5, b_4, b_3, b_2, b_1)$$

Entschlüsseln Sie die Nachricht:

$$y = 110011\ 010101\ 101100$$

mit dem jeweils geforderten Betriebsmodus und geben Sie in jeder Teilaufgabe den Plaintext x an:

- (a) **ECB Modus** (Electronic Code Book Mode)

Lösung:

Es wird blockweise entschlüsselt:

ECB: $y = 110011\ 010101\ 101100$

$$x_1 = 110011$$

$$x_2 = 010101$$

$$x_3 = 001101$$

$$x = 110011\ 010101\ 001101$$

- (b) **CBC Modus** (Cipher Block Chaining Mode) mit Initialisierungsvektor $IV = 001100$

Lösung:

CBC: $y = 110011\ 010101\ 101100$ und $IV: 001100$

$$110011 \oplus 001100 = 111111 \rightarrow d(110011) = x_1 = 111111$$

$$010101 \oplus 110011 = 011001 \rightarrow d(010101) = x_2 = 011001$$

$$001101 \oplus 010101 = 011000 \rightarrow d(101100) = x_3 = 011000$$

$$x = 111111\ 011001\ 011000$$

2. RSA Schlüsselpaare

Vervollständigen Sie das Schlüsselpaar. Ist es nicht möglich ein Schlüsselpaar zu bilden, begründen Sie warum die gewählten Parameter *ungültig* sind. Führen Sie für vollständige Schlüsselpaare bei gegebenen Klartext x eine Verschlüsselung und bei gegebenen Chiffre y eine Entschlüsselung aus. Um den fehlenden RSA-Parameter d zu bestimmen, nutzen Sie bitte den erweiterten euklidischen Algorithmus:

Algorithm 1 Der Erweiterte Euklidische Algorithmus (EEA)

Eingabe: Zwei positive Zahlen r_0 und r_1 wobei $r_0 > r_1$

Ausgabe: $ggT(r_0, r_1)$ sowie s und t mit $ggT(r_0, r_1) = s \cdot r_0 + t \cdot r_1$

Initialisierung: $s_0 = 1, s_1 = 0, t_0 = 0, t_1 = 1, i = 1$

Algorithmus:

```
1: repeat
2:    $i = i + 1$ 
3:    $r_i = r_{i-2} \bmod r_{i-1}$ 
4:    $q_{i-1} = (r_{i-2} - r_i) / r_{i-1}$ 
5:    $s_i = s_{i-2} - q_{i-1} \cdot s_{i-1}$ 
6:    $t_i = t_{i-2} - q_{i-1} \cdot t_{i-1}$ 
7: until  $r_i = 0$  {Die Schleife wird solange wiederholt bis  $r_i$  zu 0 wird.}
8: return  $ggT(r_0, r_1) = r_{i-1}$  und  $s = s_{i-1}$  und  $t = t_{i-1}$ 
```

(a) $p = 31, q = 5, e = 3, x = 42$

Lösung:

$$N = p \cdot q \equiv 31 \cdot 5 \equiv 155$$

$$T = (p - 1) \cdot (q - 1) \equiv 30 \cdot 4 \equiv 120$$

Damit die Parameter gültig sind muss $\text{ggT}(T, e) = 1$ gelten.

$$\text{Der } \text{ggT}(T, e) \equiv \text{ggT}(120, 3) = 3 \neq 1$$

Folglich sind die Parameter ungültig.

(b) $p = 83, q = 53, e = 3, x = 2$

Lösung:

$$N = p \cdot q \equiv 83 \cdot 53 \equiv 4399$$

$$T = (p - 1) \cdot (q - 1) \equiv 82 \cdot 52 \equiv 4264$$

$ggT(4264, 3) = 1$ Das sollte mit EEG gerechnet werden.

1. Lauf:

$$i = i + 1 = 2$$

$$r_2 = r_0 \bmod r_1 = 4264 \bmod 3 = 1$$

$$q_1 = (r_0 - r_2)/r_1 = (4264 - 1)/3 = 1421$$

$$s_2 = s_0 - q_1 \cdot s_1 = 1 - 1421 \cdot 0 = 1$$

$$t_2 = t_0 - q_1 \cdot t_1 = 0 - 1421 \cdot 1 = -1421$$

2. Lauf (final)

$$i = i + 1 = 3$$

$$r_3 = r_1 \bmod r_2 = 3 \bmod 1 = 0$$

$$q_2 = (r_1 - r_3)/r_2 = (3 - 0)/1 = 3$$

$$s_3 = s_1 - q_2 \cdot s_2 = 0 - 3 \cdot 1 = -3$$

$$t_3 = t_1 - q_2 \cdot t_2 = 1 - 3 \cdot (-1421) = 4264$$

Ab hier hat man alle benötigten Parameter.

Da $s \cdot T + t \cdot e$ und $t = -1421$ laut EEG, ist $d = 2843$

Kurzer Hinweis: Der private Schlüssel d entspricht dem Parameter t , wobei t die Inverse von e modulo T (Φ von n) ist. Der Parameter $t = -1421$ ist negativ, daher müssen wir den Parameter noch in den positiven Restklassenring übertragen. Daher $4264 - 1421 = 2843$. Der private Schlüssel d lautet 2843. Sie können das Ergebnis zuhause prüfen indem Sie z.B. mithilfe von WolframAlpha (<https://www.wolframalpha.com/input/>):

$3^{-1} \bmod 4264$ berechnen lassen.

Für die Verschlüsselung $y = x^e = 2^3 = 8$

3. Ein Buffer-Overflow

Der Codeausschnitt in Listing 1 beinhaltet eine Buffer-Overflow Schwachstelle auf einem **x86 32 Bit-System**. Der Buffer-Overflow ermöglicht eine erfolgreiche Authentifizierung trotz inkorrektem Passwort.

Hinweis:

Auf einem x86-32 Bit-System haben Integer und Pointer die Größe von 4 Byte. Ein char hat immer 1 Byte. Der Compiler in dieser Aufgabe führt keine Optimierungen am Code und Stack-Layout durch. Variablen werden in der gleichen Reihenfolge auf den Stack geschrieben wie sie im C-Code allokiert werden.

```
1  int check_login(char* password) {
2      int log_flag = 0;
3
4      // Allokiere Speicher auf dem Stack
5      char password_buffer[12] = {0};
6
7      // Kopiere den *gesamten* String "password" nach "password_buffer"
8      strcpy(password_buffer, password);
9
10     // Falls das eingegebene Passwort "route69" ist,
11     // setze die Variable "log_flag"
12     if (strcmp(pw_buffer, "route69") == 0)
13         log_flag = 1;
14
15     return log_flag;
16 }
17
18 int main(int argc, char *argv[]) {
19
20     if (argc != 2) {
21         printf("Benutzung: %s <password>\n", argv[0]);
22         exit(0);
23     }
24
25     // Das eingegebene Passwort wird in der Variable "pw" gespeichert.
26     char *pw = argv[1];
27
28     // Die Funktion check_login prueft, ob das richtige Passwort eingegeben wurde
29     if (check_login(pw))
30     {
31         printf("Login!\n");
32     }
33     else {
34         printf("Kein_Zugriff\n");
35     }
36 }
```

Listing 1: Buffer Overflow

- (a) Beschreiben Sie die Sicherheitslücke im Listing 1. Beschreiben Sie textuell wie diese Sicherheitslücke behoben werden kann?

Lösung:

In Zeile 8 sieht man ein `strcpy` ohne Length-Check. Der vom Angreifer kontrollierte String *password* inkl Null-Byte wird in den *password_buffer* geschrieben. Ist *password* zu lang (>12 bytes) führt das zu einem Buffer-Overflow. Man kann die Sicherheitslücke beheben, indem man:

- einen length-check vorm `strcpy` macht (verhindert den Buffer-Overflow)
- Variablen vertauschen, sodass `log_flag` unterhalb vom Buffer liegt.

- (b) Skizzieren Sie das Stackframe-Layout der Funktion `check_password` vor dem **return** statement.

Lösung:

Stack Frame vor dem Overflow:

`check_login()` frame

`&password`

return address

saved `ebp`

`log_flag`

`pw_buffer`

`strcpy()` frame

- (c) Finden Sie eine gültige Eingabe, die **nicht** *route69* ist, damit das Programm die Nachricht *Login!* ausgibt. Warum funktioniert Ihr Angriff?

Lösung:

Die Eingabe besteht aus:

- 12 beliebige Zeichen um den Buffer zu füllen.
- 1 beliebiges Zeichen um *log_flag* ungleich 0 zu setzen.

Eingaben größer 16 Byte fangen an den *ebp* Pointer zu überschreiben. Das hat in diesem Fall keine Auswirkung, solange das *return* noch stimmt.

Eine mögliche Eingabe ist folglich *AAAA AAAA AAAA BBB* oder *AAAA AAAA AAAA BBBB BB ...*

Der Angriff funktioniert, weil die Variable *log_flag* direkt oberhalb des Buffers liegt und bei einem Buffer-Overflow überschrieben wird.

In C werden Integer > 0 , als *true* gewertet. Damit kann *auth_flag* mit einem beliebigen Wert größer 0 überschrieben werden.

4. Multics

Im Folgenden wird die gleiche Notation wie in der Vorlesung verwendet.

- Ring Bracket ist gegeben als (r_1, r_2) , (r_2, r_3)
- Access Bracket (r_1, r_2) , wobei r_1 das Schreibrecht und r_2 das Leserecht definiert
- Call Bracket (r_2, r_3) , wobei r_2 das Leserecht und r_3 das Ausführungsrecht definiert
- Aktuelle Ringnummer eines Prozesses ist r
- Bei einer *Ring Transition* wechselt der Prozess in Ring r'
- Eine User-ID in Multics wird wie folgt definiert: `Person.Project.Tag`
 - Person: Name des Users
 - Project: Gruppe des Users
 - Tag: Art der Tätigkeit des Users: (m) interaktiv, (a) batch
- Rechte sind gegeben als:
 - r: read / Leserecht
 - w: write / Schreibrecht
 - e: execute / Ausführungsrecht
 - n: null / keine Rechte

Es seien die folgenden Segmente mit zugehörigen ACLs gegeben:

Segment 1

Ring Bracket: (2,4), (4,4)

```
re Martin.*.a
n *.students.*
re Manuela.students.*
```

Segment 2

Ring Bracket: (0,0), (0,5)

```
re *.students.*
rwe Maria.researchers.*
rwe Max.students.m
```

Nun möchten die unten stehenden Prozesse mit den angegebenen Usern die aufgeführten Zugriffe durchführen. Bestimmen und begründen Sie für die folgenden Fälle, ob der angefragte Zugriff durch ACL und Ringe erlaubt ist oder nicht. Falls der Zugriff erlaubt ist geben Sie auch an, in welchen Ring der Prozess wechseln muss oder ob er in seinem bisherigen Ringen weiterlaufen kann.

1. `Martin.students.a`: P_1 ($r = 2$) lesen von Segment 1

Zugriff durch ACL ☐ erlaubt ☐ verbotenZugriff durch Ring ☐ erlaubt ☐ verboten☐ Ring Switch nach _____**Lösung:**Lösung: Zugriff erlaubt. ACL erlaubt. Ring erlaubt, da $r=4 \leq 4$.

2. `Maria.students.m`: P_2 ($r = 0$) schreiben von Segment 2

Zugriff durch ACL ☐ erlaubt ☐ verbotenZugriff durch Ring ☐ erlaubt ☐ verboten☐ Ring Switch nach _____**Lösung:**Lösung: Zugriff verboten. ACL verboten. Ring erlaubt, da $r=0 \leq 0$.

3. `Manuela.students.m`: P_3 ($r = 3$) ausführen von Segment 1

Zugriff durch ACL ☐ erlaubt ☐ verbotenZugriff durch Ring ☐ erlaubt ☐ verboten☐ Ring Switch nach _____**Lösung:**Lösung: Zugriff erlaubt. ACL erlaubt. Ring erlaubt, da $r=3 \leq 4$. Keine Ring transition erforderlich.

4. `Martin.students.a`: P_4 ($r = 0$) ausführen von Segment 1

Zugriff durch ACL ☐ erlaubt ☐ verbotenZugriff durch Ring ☐ erlaubt ☐ verboten☐ Ring Switch nach _____**Lösung:**Lösung: Zugriff erlaubt. ACL erlaubt. Ring erlaubt, da $r=0 \leq 2$. Ring transition nach 2.

5. Mareike.students.a: P_5 ($r = 4$) ausführen von Segment 2

Zugriff durch ACL ☐ erlaubt ☐ verboten

Zugriff durch Ring ☐ erlaubt ☐ verboten

☐ Ring Switch nach _____

Lösung:

Lösung: Zugriff erlaubt. ACL erlaubt. Ring erlaubt da $r=4 \leq 5$. Ring transition zu $r=0$.

6. Maria.researchers.m: P_6 ($r = 1$) schreiben von Segment 2

Zugriff durch ACL ☐ erlaubt ☐ verboten

Zugriff durch Ring ☐ erlaubt ☐ verboten

☐ Ring Switch nach _____

Lösung:

Lösung: Zugriff verboten. ACL erlaubt. Ring verboten, da $r=1 > 0$.